DS-20250919-CatBoost: Unbiased Boosting with Categorical Features

Michael.Niu, Haoxing.Liu, Kailun.Dong

Sep 19, 2025

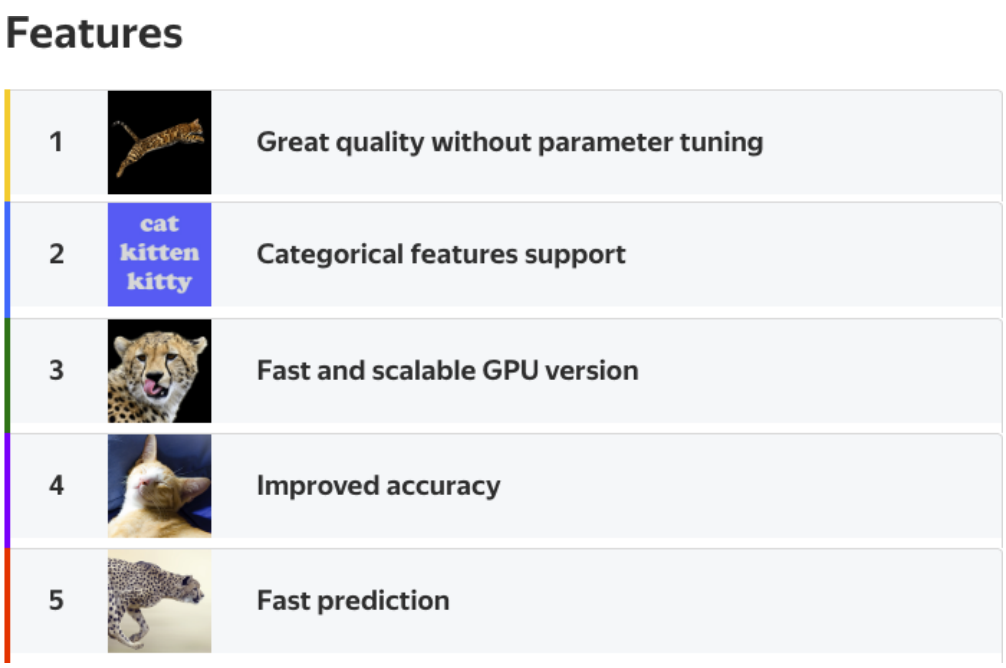


Figure 1: Not just CATegorical, but CATs too!

1 Overview

CatBoost is a gradient boosting algorithm that is particularly powerful when dealing with datasets that have a lot of categorical features. Typically, when dealing with categorical data, some methods to convert these categorical features into numerical values are needed. The CatBoost algorithm introduces a novel technique called **Ordered Target Statistics** to convert categorical features into numerical values without the common issue of target leakage. It also proposes a unique training scheme called **Ordered Boosting** to prevent prediction shift or bias towards training data. Engineering-wise, CatBoost chooses an efficient implementation based on a **Symmetric Decision Tree**, where at each depth level, every node uses the same split. Additionally, CatBoost automatically generates combinations of categorical features to capture complex interactions in the data. This combination of techniques makes it a highly accurate and robust algorithm right out of the box.

2 Boosting

2.1 Adaboost

AdaBoost (Adaptive Boosting) is a powerful and popular ensemble learning algorithm. It works by combining a series of "weak learners," typically simple decision trees, to create a single "strong learner." The algorithm is adaptive because it iteratively adjusts the weights of the training data instances, giving more importance to those that were misclassified by previous weak learners. The goal of each subsequent learner is to focus on and correctly classify the difficult instances.

The mathematical foundation of AdaBoost can be understood as a forward stagewise additive modeling procedure that minimizes a specific loss function.

2.2 Loss Functions in AdaBoost

The choice of loss function is what gives AdaBoost its unique properties, particularly its ability to sequentially build a strong classifier. For binary classification problems with class labels $y \in \{-1, 1\}$, the model's output at stage m is represented as $f_m(x)$, which is an additive combination of the previous model $f_{m-1}(x)$ and a new weak learner $G_m(x)$ with a weight β_m.

$$f_m(x) = f_{m-1}(x) + \beta_m G_m(x)$$

The central task at each step is to find the best weak learner G_m and its coefficient β_m that minimize the overall loss.

$$(\beta_m, G_m) = \arg\min_{\beta, G} \sum_{i=1}^{N} L(y_i, f_{m-1}(x_i) + \beta G(x_i))$$

2.3 Exponential Loss

The standard AdaBoost.M1 algorithm, as referenced in the provided document, uses the **exponential loss function**. This loss function is defined as:

$$L(y, f(x)) = \exp(-yf(x)) \quad (10.8)$$

This function penalizes misclassifications exponentially. If a data point is correctly classified (y and $f(x)$ have the same sign), the loss is close to zero. If it is misclassified (y and $f(x)$ have opposite signs), the loss grows exponentially, making the algorithm highly sensitive to misclassified points.

The minimization objective for exponential loss at each stage m can be written as:

$$(\beta_m, G_m) = \arg\min_{\beta, G} \sum_{i=1}^{N} \exp[-y_i(f_{m-1}(x_i) + \beta G(x_i))] \quad (10.9)$$

This can be simplified by recognizing that the term $\exp(-y_i f_{m-1}(x_i))$ is constant at step m and can be treated as a weight $w_i^{(m)}$ for each observation, as shown in the attached image.

$$w_i^{(m)} = \exp(-y_i f_{m-1}(x_i))$$

The minimization problem then becomes:

$$(\beta_m, G_m) = \arg\min_{\beta, G} \sum_{i=1}^{N} w_i^{(m)} \exp[-y_i \beta G(x_i)]$$

The optimal weak learner G_m and coefficient β_m can be derived by minimizing this weighted loss. This leads to the key update rules of the AdaBoost algorithm, where the weights of misclassified instances are increased exponentially.

2.4 Cross-Entropy (Deviance) Loss

While the original AdaBoost algorithm is based on exponential loss, modern boosting algorithms like Gradient Boosting often use **cross-entropy loss**, also known as binomial deviance, particularly for classification problems. This alternative perspective highlights the connection between boosting and statistical models.

$$L(y, p) = -[y \log(p) + (1 - y) \log(1 - p)]$$

where p is the predicted probability, typically given by the sigmoid function of the model's output, $p = \frac{1}{1+e^{-f(x)}}$.

The exponential loss function is a more aggressive upper bound of the binomial deviance loss. While both aim to separate classes and penalize misclassifications, the exponential loss places a much higher penalty on large negative margins (i.e., highly confident but incorrect predictions), making AdaBoost more susceptible to noisy data or outliers compared to boosting algorithms that use deviance loss. This is one of the key reasons why gradient boosting, which uses a variety of loss functions including deviance, has become a more flexible and robust alternative.

2.5 Gradient Boosting

2.5.1 Comparison to AdaBoost

Unlike AdaBoost, which uses an explicit exponential loss and updates sample weights, Gradient Boosting provides a more general framework. It can be applied to any differentiable loss function (e.g., squared loss or deviance). For squared error loss, this process simplifies to simply fitting a regression tree to the current residuals, $y_i - f_{m-1}(x_i)$. For other loss functions, the "residuals" are the negative gradients. This means you can apply the boosting framework to a much wider range of problems, including regression (using Mean Squared Error or Mean Absolute Error) and different types of classification problems (using deviance/log-loss).

Gradient Boosting is a popular method that builds trees step-by-step by fitting each new tree to the *residual errors* (gradients) of the previous model. Each new tree $f_t(x)$ acts as a corrective function to reduce the residual error of the current ensemble. The learning rate η prevents overfitting by shrinking the contribution of each tree.

2.5.2 Model Update Rule

Let $F_t(x)$ denote the model prediction after t rounds of boosting. The core update formula is:

$$F_t(x) = F_{t-1}(x) + \eta f_t(x) \tag{1}$$

where

- $F_{t-1}(x)$ is the ensemble model after $t - 1$ rounds,
- $f_t(x)$ is the new weak learner (usually a shallow decision tree),
- $\eta \in (0, 1]$ is the learning rate (shrinkage factor) controlling the contribution of $f_t(x)$.

2.5.3 Algorithm Steps

1. Initialize the model with a constant prediction, e.g.,

$$F_0(x) = \arg\min_{\gamma} \sum_{i=1}^{n} l(y_i, \gamma)$$

2. For $t = 1$ to T:

(a) Compute gradients (residuals):

$$g_i = \left. \frac{\partial l(y_i, \hat{y}_i)}{\partial \hat{y}_i} \right|_{\hat{y}_i = F_{t-1}(x_i)}$$

and optionally the Hessians for second-order approximation:

$$h_i = \left. \frac{\partial^2 l(y_i, \hat{y}_i)}{\partial \hat{y}_i^2} \right|_{\hat{y}_i = F_{t-1}(x_i)}$$

(b) Fit a new decision tree $f_t(x)$ to predict g_i (or negative gradients).

(c) Update the ensemble:

$$F_t(x) = F_{t-1}(x) + \eta f_t(x)$$

3. Return $F_T(x)$ as the final model.

2.5.4 Core Concept: Forward Stagewise Additive Modeling

The central idea behind Gradient Boosting is to construct a predictive model $f_M(x)$ as a sum of simpler, "weak" learners, denoted as $T(x; \Theta_m)$. This is a forward stagewise manner, where at each iteration m, a new tree is added to the model:

$$f_m(x) = f_{m-1}(x) + T(x; \Theta_m)$$

The goal at each step is to find the parameters Θ_m of the next tree that minimize the overall loss function L on the training data:

$$\hat{\Theta}_m = \arg\min_{\Theta_m} \sum_{i=1}^{N} L(y_i, f_{m-1}(x_i) + T(x_i; \Theta_m)) \quad (10.29)$$

This is a very greedy strategy, as it aims to maximally reduce the loss at each step.

2.5.5 Gradient Boosting as Numerical Optimization

Gradient Boosting views the model-building process as a numerical optimization problem. The objective is to minimize the total loss on the training data, $L(f) = \sum_{i=1}^{N} L(y_i, f(x_i))$, where $f(x)$ is the entire function to be optimized.

This minimization is performed using a technique analogous to **steepest descent**. In numerical optimization, a "step" is taken in the direction of the negative gradient of the loss function. The gradient vector $\mathbf{g}_m$ has components for each data point i given by:

$$g_{im} = \left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)} \right]_{f(x_i) = f_{m-1}(x_i)} \quad (10.35)$$

At each iteration, the model is updated by taking a step in the direction of this negative gradient:

$$\mathbf{f}_m = \mathbf{f}_{m-1} - \rho_m \mathbf{g}_m$$

where ρ_m is the step length, determined by a line search to minimize the loss.

2.5.6 Fitting Trees to the Negative Gradient

A key challenge is that the gradient is a set of values for the training data, not a function that can be generalized to new data. To overcome this, instead of directly using the gradient vector $\mathbf{g}_m$, Gradient Boosting induces a regression tree $T(x; \Theta)$ that **approximates the negative gradient**. This is done by fitting the tree to the negative gradient values using squared error as the measure of closeness:

$$\hat{\Theta}_m = \arg\min_{\Theta} \sum_{i=1}^{N} \left(-g_{im} - T(x_i; \Theta) \right)^2 \quad (10.37)$$

This means that at each iteration, a regression tree is built to predict the negative gradient of the loss function. This tree learns to correct the errors made by the current model.

2.5.7 Gradient Descent in Traditional ML vs. Gradient Boosting

The fundamental differences between the application of gradient descent in traditional machine learning models (e.g., neural networks) and in the boosting framework (specifically, gradient boosting) lies in the space where the optimization occurs: parameter space versus function space.

Traditional Gradient Descent	Gradient Boosting
Optimization Space Operates in **parameter space**. The goal is to find the optimal values for a fixed set of model parameters, $\boldsymbol{\theta}$.	**Optimization Space** Operates in **function space**. The goal is to find the best additive function, $F_m(\mathbf{x})$, by iteratively adding a new weak learner.
What is Minimized? Minimizes the loss function $L(\boldsymbol{\theta})$ with respect to the model's parameters.	**What is Minimized?** Minimizes the overall loss of the ensemble, $L(F_{m-1}(\mathbf{x}) + \rho_m h_m(\mathbf{x}))$, with respect to the new function $h_m(\mathbf{x})$ and its coefficient ρ_m.
Gradient Calculation The gradient is calculated with respect to the model's parameters, $\nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta})$.	**Gradient Calculation** The gradient is calculated with respect to the model's predictions, $\frac{\partial L(y_i, F(\mathbf{x}_i))}{\partial F(\mathbf{x}_i)}$. This is also called the "pseudo-residual."
Model Update Rule Updates the parameters in the negative gradient direction, scaled by a learning rate α: $\boldsymbol{\theta}_{\text{new}} = \boldsymbol{\theta}_{\text{old}} - \alpha \nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta})$	**Model Update Rule** Adds a new weak learner ($h_m(\mathbf{x})$) that is trained to fit the negative gradient (pseudo-residuals). The model is updated as: $F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \rho_m h_m(\mathbf{x})$

2.5.8 Summary of Key Differences

- **Objective**: Traditional gradient descent adjusts a single, predefined function. Gradient boosting builds an ensemble of simple functions.
- **Iterative Step**: In each iteration, traditional methods update a set of numbers (parameters). Boosting adds a new function to the model.
- **Generalization**: Gradient boosting is a more general framework. It can be applied to any differentiable loss function and uses a variety of weak learners, making it highly flexible.

2.5.9 Gradient Tree Boosting Algorithm

Algorithm 1 Gradient Tree Boosting Algorithm

Initialize $f_0(x) = \arg\min_\gamma \sum_{i=1}^{N} L(y_i, \gamma)$.

for $m = 1$ to M **do**

For $i = 1, 2, \ldots, N$ compute the pseudo-residuals:

$$r_{im} = -\left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)}\right]_{f=f_{m-1}}.$$

Fit a regression tree to the targets r_{im}, giving terminal regions R_{jm}, $j = 1, 2, \ldots, J_m$.

for $j = 1, 2, \ldots, J_m$ **do**

Compute the optimal output value γ_{jm} for each region:

$$\gamma_{jm} = \arg\min_\gamma \sum_{x_i \in R_{jm}} L(y_i, f_{m-1}(x_i) + \gamma).$$

Update: $f_m(x) = f_{m-1}(x) + \sum_{j=1}^{J_m} \gamma_{jm} I(x \in R_{jm})$.

Output the final model $\hat{f}(x) = f_M(x)$. =0

3 XGBoost: Detailed Improvements over Gradient Boosting

XGBoost is an enhanced gradient boosting implementation that introduces regularization, second-order optimization, and efficient engineering. Below are the main improvements explained mathematically. Before XGBoost, Friedman's GBM and its R/Python implementations were the standard. XGBoost took this foundation and added

- regularization
- second-order optimization
- sparsity handling
- highly efficient parallel computation, which made it much faster and more accurate for large datasets.

3.1 Regularized Objective Function

Unlike classical gradient boosting, XGBoost optimizes a regularized objective:

$$\mathcal{L}^{(t)} = \sum_{i=1}^{n} l(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t), \tag{2}$$

where the tree complexity regularizer is

$$\Omega(f) = \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2 + \alpha \sum_{j=1}^{T} |w_j|, \tag{3}$$

with:

- T = number of leaves in the tree,
- w_j = weight of leaf j,
- γ = penalty for adding a new leaf,

- λ = L2 regularization on leaf weights,
- α = L1 regularization on leaf weights.

3.2 Second-Order Approximation

XGBoost uses second-order Taylor expansion of the loss to compute optimal leaf weights efficiently:

$$l(y_i, \hat{y}_i + f_t(x_i)) \approx l(y_i, \hat{y}_i) + g_i f_t(x_i) + \frac{1}{2} h_i f_t(x_i)^2, \tag{4}$$

where

$$g_i = \frac{\partial l(y_i, \hat{y}_i)}{\partial \hat{y}_i}, \quad h_i = \frac{\partial^2 l(y_i, \hat{y}_i)}{\partial \hat{y}_i^2}.$$

3.3 Optimal Leaf Weights

For leaf j with samples I_j, define

$$G_j = \sum_{i \in I_j} g_i, \quad H_j = \sum_{i \in I_j} h_i.$$

Then the optimal leaf weight minimizing the regularized objective is:

$$w_j^* = \begin{cases} -\frac{G_j}{H_j + \lambda}, & \text{if } \alpha = 0 \text{ (L2 only)} \\ -\frac{\text{sign}(G_j) \max(|G_j| - \alpha, 0)}{H_j + \lambda}, & \text{if } \alpha > 0 \text{ (L1 + L2)} \end{cases} \tag{5}$$

3.4 Split Gain

The split gain is a metric used to evaluate a potential split at a node. The algorithm considers all possible splits and chooses the one with the highest positive gain. If the best possible gain is not positive, no split is made, and the node becomes a leaf.

To decide the best split at a node, XGBoost computes the regularized gain:

$$\text{Gain} = \frac{1}{2} \left(\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right) - \gamma, \tag{6}$$

where L and R denote left and right child nodes after the split. A split is only applied if Gain > 0.

The first two terms, $\frac{G_L^2}{H_L+\lambda}$ and $\frac{G_R^2}{H_R+\lambda}$, represent the loss from the left and right child nodes after the split. The third term, $\frac{(G_L+G_R)^2}{H_L+H_R+\lambda}$, represents the loss of the parent node before the split. The final term, γ, is the regularization penalty for adding a new leaf.

3.5 Algorithmic Enhancements

These improvements focus on the core mathematics of the boosting process to enhance accuracy and control overfitting.

- **Second-Order Gradients:** XGBoost uses the second-order derivative (Hessian) of the loss function in addition to the first-order gradient. This provides more accurate information about the direction and magnitude of the loss, leading to a more precise optimization for finding optimal leaf weights.
- **Regularized Objective Function:** The algorithm explicitly incorporates **L1/L2 penalties** on the leaf weights and a penalty (γ) on the number of leaves directly into its objective function. This actively prevents overfitting and enables the calculation of **regularized split gain**, ensuring a new split is made only if the gain in performance outweighs the cost of complexity.

- **Shrinkage:** Predictions are updated gradually using a **learning rate** (η). This reduces the impact of each new tree, allowing the model to make more conservative, stable updates and preventing it from overfitting to a specific training instance. The update rule is:

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + \eta f_t(x_i)$$

3.6 Engineering Optimizations

These improvements are focused on computational efficiency and handling real-world data at scale.

- **Sparsity Awareness:** The algorithm is designed to natively handle missing data by learning the optimal default direction for a split (left or right branch) for missing values.
- **Approximate Split Finding:** For very large datasets that cannot fit into memory, XGBoost uses a histogram-based, approximate algorithm to find the best split points efficiently, dramatically speeding up training without sacrificing much accuracy.
- **Parallel Computing:** XGBoost supports parallel and distributed computing for training. It parallelizes the most time-consuming part of tree building—the process of finding the best splits for all features—by distributing the data across multiple CPU cores or machines.

4 LightGBM: Further Improvements

LightGBM is another gradient boosting implementation designed for large-scale and high-dimensional datasets. Key innovations:

1. **Leaf-wise growth:** Unlike level-wise growth in traditional GBM, LightGBM grows the leaf with the largest loss reduction, allowing deeper trees with fewer splits.
2. **Histogram-based binning:** Discretizes continuous features into bins, reducing memory usage and speeding up split computation.
3. **Gradient-based One-Side Sampling (GOSS):** Uses samples with large gradients fully and randomly subsamples small-gradient samples to accelerate training.
4. **Exclusive Feature Bundling (EFB):** Bundles mutually exclusive sparse features to reduce feature dimensionality.

5 Categorical Features and Target Statistics(TS)

A distinctive challenge in gradient boosting is the handling of categorical features, especially those with high cardinality (e.g., user identifiers, product IDs). Standard one-hot encodings are infeasible due to dimensionality, while naïve numerical encodings fail to capture statistical meaning. A common solution is to replace a categorical value by a **target statistic (TS)**, i.e. an empirical estimate of the conditional expectation of the target given the category.

5.1 Traditional Target Statistics

Let $x^{(j)}$ denote a categorical feature. y_i denotes the target values, and c a category value. Target Statistic (TS) is a numeric encoding of the question "if the category is c, what is the average target value among samples with this category?" A simple estimator for regression is

$$\mathrm{TS}(c) = \frac{\sum_{i:x_i^{(j)}=c} y_i}{\#\{i : x_i^{(j)} = c\}}, \tag{7}$$

i.e. the mean target for all samples belonging to category c. In practice, to reduce variance and avoid extreme values for rare categories, one introduces smoothing with a prior μ and parameter $a > 0$:

$$\mathrm{TS}(c) = \frac{\sum_{i:x_i^{(j)}=c} y_i + a\mu}{\#\{i : x_i^{(j)} = c\} + a}. \tag{8}$$

While effective, this formulation suffers from *target leakage*: the value y_i of the current training sample contributes directly to the statistic used as a feature for predicting y_i itself. This induces a distributional mismatch between training and test features. Alternative fixes such as leave-one-out (removing y_i from the sum) or holdout-based TS (computing TS on a data split) partially alleviate leakage, but either still bias the estimator or waste available data.

5.2 General Approaches to Avoid Target Leakage

Several strategies have been proposed to remove the direct contribution of y_i from the statistic used to encode sample i:

- **Leave-one-out (LOO).** For each sample i, compute the statistic using all other samples but exclude i. This prevents exact leakage but still creates bias in degenerate cases. For example, if all samples of a category share the same label, the leave-one-out statistic perfectly predicts the left-out sample.
- **Holdout encoding.** Split the training set into two disjoint parts: one to compute target statistics and the other to train the model. This removes leakage but wastes data, since each split only uses part of the available information.
- **K-fold or cross-validated encoding.** Partition the training set into K folds, and compute the statistic for each fold using the remaining $K-1$ folds. This uses more data than holdout encoding but still requires repeated data passes and can introduce variance between folds.

All of these methods belong to the general idea of computing TS without directly using the target y_i of the current sample. This principle can be written in the following general form:

$$\hat{f}(x_k) = \frac{\sum_{(x,y)\in D_k,\, x^{(j)}=x_k^{(j)}} y}{\#\{(x,y) \in D_k : x^{(j)} = x_k^{(j)}\}}, \quad \text{where } D_k \subseteq D,\ (x_k, y_k) \notin D_k. \tag{9}$$

Here, D is the training dataset and D_k is the dataset with the k-th sample removed. This construction guarantees that the statistic used to encode x_k does not depend on y_k, thus eliminating target leakage in principle. However, practical implementations such as pure leave-one-out or holdout may still suffer from high variance or inefficient data usage. These methods all aim to reduce or eliminate the dependence of $\mathrm{TS}(c)$ on the target y_i of the current sample. However, they trade off between bias, variance, and data efficiency.

5.3 Ordered Target Statistics in CatBoost

CatBoost addresses this issue with the concept of *ordered target statistics*. The key idea is to simulate an online learning process in which each sample is encoded using only information available from "past" samples, thus ensuring that the statistic is conditionally independent of its own target.

Formally, fix a random permutation σ of the training dataset. For an example i located at position $p = \sigma(i)$, the CatBoost target statistic is defined as

$$\mathrm{TS}_i(c) = \frac{\sum_{k:\,\sigma(k)<p,\, x_k^{(j)}=c} y_k + a\mu}{\#\{k :\, \sigma(k) < p,\, x_k^{(j)} = c\} + a}. \tag{10}$$

Thus, only samples preceding i in the permutation are used to compute the statistic for i.

This construction has two essential properties:

1. **No leakage.** By design, y_i never influences $\mathrm{TS}_i(c)$, eliminating the conditional shift between training and test.

2. **Full data efficiency.** Every sample participates in the encoding of later samples, avoiding the data wastage inherent in holdout-based methods.

5.4 Training vs. Inference Usage

A crucial distinction is that ordered TS is only used during training to prevent leakage. In particular:

- During training, each sample receives a *different* encoding value $\mathrm{TS}_i(c)$ depending on its position in the permutation. As a result, even samples with the same category may take different feature values and may be routed differently through tree splits. This prevents the model from "cheating" by memorizing its own target.

- During inference, there is no leakage risk because the test target y_{test} is unknown. Therefore, each category c is assigned a *single stable TS value* computed using the entire training dataset:

$$\mathrm{TS}(c) = \frac{\sum_{i:x_i^{(j)}=c} y_i + a\mu}{\#\{i : x_i^{(j)} = c\} + a}. \tag{11}$$

  This ensures consistent behavior for all test samples belonging to the same category.

Thus, the training phase learns unbiased split thresholds and leaf updates using ordered TS, while the inference phase applies those trained rules to features encoded by full-data TS.

5.5 Summary of Improvements

Compared to traditional target statistics, CatBoost's ordered formulation:

- Removes prediction bias due to target leakage.

- Preserves the effective sample size by reusing the full dataset.

- Provides stable encodings for high-cardinality features through permutation averaging and smoothing.

- Ensures consistent category encodings at inference, while preserving unbiased training dynamics.

6 Prediction Shift and Ordered Boosting

6.1 Gradient Boosting with Residuals

Let the training dataset be

$$D = \{(x_k, y_k)\}_{k=1}^n,$$

where $x_k \in \mathbb{R}^d$ may include both numerical and categorical features (the latter possibly encoded as target statistics). The boosting model at iteration t is an additive function

$$f_{t+1}(x) = f_t(x) + \eta h_t(x), \tag{12}$$

with learning rate η and weak learner h_t (typically a tree).

At each iteration, we form pseudo-residuals by differentiating the loss:

$$r_k^{(t)} = -\frac{\partial L(y_k, f_t(x_k))}{\partial f_t(x_k)}. \tag{13}$$

Each residual $r_k^{(t)}$ is a scalar. Collecting them gives the residual vector

$$r^{(t)} = \left(r_1^{(t)}, r_2^{(t)}, \ldots, r_n^{(t)}\right)^\top \in \mathbb{R}^n.$$

At iteration t, the weak learner h_t is trained as a regression function mapping features to residuals:

$$h_t = \arg\min_h \sum_{k=1}^{n} \left(r_k^{(t)} - h(x_k)\right)^2.$$

Equivalently, the dataset for training h_t is

$$\{(x_k, r_k^{(t)})\}_{k=1}^n.$$

Thus the n residuals are put together as the *target vector* of a supervised regression problem, with the original features x_k as inputs.

6.2 The Problem of Prediction Shift

In principle, $r_k^{(t)}$ should represent the conditional gradient distribution on unseen test samples $(x, y) \sim \mathcal{D}$. However, in standard boosting, residuals are computed using the same dataset D on which f_t was trained. This creates a dependency:

$$r_k^{(t)} \mid (x_k, y_k) \in D \quad \neq \quad r(x, y) \mid (x, y) \sim \mathcal{D}.$$

Intuitively, f_t has already adapted to (x_k, y_k), so the residual for that sample is systematically smaller or shifted compared to a fresh test point. When h_t is fit on these shifted residuals, its split selection and leaf values are biased. This distributional mismatch is called *prediction shift*.

6.3 General Approaches to Avoid Prediction Shift

Several strategies have been proposed:

- **Independent datasets.** Use one dataset to compute residuals and another to fit weak learners. Eliminates dependency but wastes data.
- **Cross-fitting.** Partition into folds; compute residuals for each fold using models trained on the remaining folds. More efficient, but computationally heavy.
- **Bagging or subsampling.** Randomly subsample when training h_t. Reduces overfitting, but residuals are still influenced by overlapping training points.

6.4 Ordered Boosting in CatBoost

CatBoost introduces *ordered boosting*, which reframes training as an online process. Fix a random permutation σ of $\{1, \ldots, n\}$. For sample k placed at position $p = \sigma(k)$, define a prefix dataset

$$D_{p-1} = \{(x_{\sigma(j)}, y_{\sigma(j)}) : j < p\}.$$

Residuals are then computed as

$$r_{\sigma(p)}^{(t)} = -\frac{\partial L(y_{\sigma(p)}, f_{t,p-1}(x_{\sigma(p)}))}{\partial f(x_{\sigma(p)})}, \tag{14}$$

where $f_{t,p-1}$ is the model trained up to iteration t using only the prefix D_{p-1}.

Thus each sample's residual is computed as if it were a *new test point* arriving after the prefix. This ensures $r_{\sigma(p)}^{(t)}$ is conditionally independent of $(x_{\sigma(p)}, y_{\sigma(p)})$ being in the training set, removing prediction shift.

6.5 Complexity Considerations

At first glance, ordered boosting appears to increase complexity dramatically. Conceptually:

- In standard boosting, we compute n residuals at iteration t, each using the same model f_t.
- In ordered boosting, $r^{(t)}_{\sigma(p)}$ depends on $f_{t,p-1}$, which itself is trained on a different prefix D_{p-1}.

This suggests that computing all residuals $\{r^{(t)}_{\sigma(p)}\}_{p=1}^{n}$ requires training n separate models, effectively turning an $O(n)$ problem into an $O(n^2)$ one (since each prefix grows as p increases).

However, CatBoost avoids this blow-up by exploiting the structure of decision trees:

- The tree *structure* (splits) can be shared across all prefix models.
- Only the *leaf values* differ depending on prefix size, and these can be updated incrementally as new samples are added.

Thus, while ordered boosting is conceptually an $n(n-1)/2$-sized problem, CatBoost implements it in a way whose practical complexity is comparable to standard gradient boosting.

6.6 Training vs. Inference Usage

During training:

- Each residual $r^{(t)}_{\sigma(p)}$ is computed using prefix models to avoid leakage.
- The weak learner h_t is then fit on the unbiased set of pairs $\{(x_{\sigma(p)}, r^{(t)}_{\sigma(p)})\}$.

During inference:

- The final ensemble f_T is applied directly to new samples.
- No permutation is needed, since test targets are unknown and there is no leakage risk.

6.7 Summary of Improvements

Compared to classical boosting, ordered boosting:

- Fits each h_t on residuals that are conditionally independent of the sample's inclusion in training.
- Preserves full data efficiency by reusing the dataset across permutations.
- Achieves computational feasibility by sharing tree structures across prefix models.
- Removes prediction shift and aligns the empirical training distribution with the true test distribution.

7 The CatBoost Algorithm: Implementation Details

The previous sections explained the conceptual innovations of CatBoost: target statistics for categorical features and ordered boosting to prevent prediction shift. We now turn to the implementation, which makes these ideas computationally efficient.

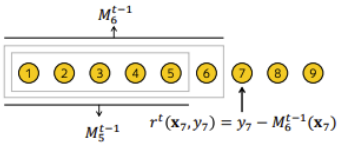


Figure 1: Ordered boosting principle, examples are ordered according to σ.

Algorithm 1: Ordered boosting

input : $\{(\mathbf{x}_k, y_k)\}_{k=1}^n$, I;

$\sigma \leftarrow$ random permutation of $[1, n]$;
$M_i \leftarrow 0$ for $i = 1..n$;
for $t \leftarrow 1$ **to** I **do**
  for $i \leftarrow 1$ **to** n **do**
    $r_i \leftarrow y_i - M_{\sigma(i)-1}(\mathbf{x}_i)$;
  for $i \leftarrow 1$ **to** n **do**
    $\Delta M \leftarrow$ $LearnModel((\mathbf{x}_j, r_j) : \sigma(j) \leq i)$;
    $M_i \leftarrow M_i + \Delta M$;
return M_n

Algorithm 2: Building a tree in CatBoost

input : M, $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$, α, L, $\{\sigma_i\}_{i=1}^s$, $Mode$

$grad \leftarrow CalcGradient(L, M, y)$;
$r \leftarrow random(1, s)$;
if $Mode = Plain$ **then**
  $G \leftarrow (grad_r(i)$ for $i = 1..n)$;
if $Mode = Ordered$ **then**
  $G \leftarrow (grad_{r,\sigma_r(i)-1}(i)$ for $i = 1..n)$;
$T \leftarrow$ empty tree;
foreach *step of top-down procedure* **do**
  foreach *candidate split* c **do**
    $T_c \leftarrow$ add split c to T;
    if $Mode = Plain$ **then**
      $\Delta(i) \leftarrow$ avg$(grad_r(p)$ for $p : leaf_r(p) = leaf_r(i))$ for $i = 1..n$;
    if $Mode = Ordered$ **then**
      $\Delta(i) \leftarrow$ avg$(grad_{r,\sigma_r(i)-1}(p)$ for $p : leaf_r(p) = leaf_r(i), \sigma_r(p) < \sigma_r(i))$ for $i = 1..n$;
    $loss(T_c) \leftarrow \cos(\Delta, G)$
  $T \leftarrow \arg\min_{T_c}(loss(T_c))$
if $Mode = Plain$ **then**
  $M_{r'}(i) \leftarrow M_{r'}(i) - \alpha$ avg$(grad_{r'}(p)$ for $p : leaf_{r'}(p) = leaf_{r'}(i))$ for $r' = 1..s, i = 1..n$;
if $Mode = Ordered$ **then**
  $M_{r',j}(i) \leftarrow M_{r',j}(i) - \alpha$ avg$(grad_{r',j}(p)$ for $p : leaf_{r'}(p) = leaf_{r'}(i), \sigma_{r'}(p) \leq j)$ for $r' = 1..s$, $i = 1..n$, $j \geq \sigma_{r'}(i) - 1$;
return T, M

Figure 2: Main Algorithms Sketch

7.1 High-Level Algorithm Flow

At a high level, CatBoost performs the following steps:

1. **Input.** Training dataset $D = \{(x_k, y_k)\}_{k=1}^n$, learning rate η, maximum depth d, number of iterations T.

2. **Preprocessing.** For each categorical feature, compute ordered target statistics using one or more random permutations of D. Multiple permutations are employed to reduce variance:

   - For categorical encodings, statistics from different permutations can either be averaged to produce a stable single feature or used as multiple parallel encodings.
   - For residuals in ordered boosting, CatBoost typically selects one permutation per boosting iteration (or cycles among them). This avoids explicit averaging but spreads the variance across iterations, so the randomness cancels out in expectation.

3. **Initialization.** Set base model $f_0(x)$ (typically constant prediction, e.g. average target).

4. **Boosting loop (for $t = 1, \ldots, T$):**

   (a) Compute pseudo-residuals $r_k^{(t)}$ for all training points using ordered boosting (prefix models with respect to the chosen permutation).

   (b) Train a new decision tree h_t on dataset $\{(x_k, r_k^{(t)})\}_{k=1}^n$:

      i. Grow a symmetric binary tree of depth d.

ii. At each split, choose the feature and threshold maximizing reduction in loss.

(c) Compute leaf values of h_t by solving a regularized least squares problem using prefix-dependent statistics.

(d) Update model: $f_{t+1}(x) = f_t(x) + \eta h_t(x)$.

5. **Output.** Final model $f_T(x)$.

7.2 Symmetric Tree Design

CatBoost restricts weak learners to *symmetric (oblivious)* trees. A symmetric tree of depth d has exactly 2^d leaves and the same split condition applied across all nodes at each level. Concretely:

- At depth 1, a single feature split is chosen and applied to the entire dataset.
- At depth 2, another feature split is applied at the second level, identically for all nodes.
- Continuing to depth d, every path follows the same sequence of d binary split questions.

This structure implies that each sample's path through the tree can be represented as a d-bit binary string (yes/no decisions). The leaf index is simply the integer value of this binary string, so prediction requires exactly d feature checks per tree.

Advantages over regular trees.

- **Fast evaluation.** No variable path lengths; each prediction requires exactly d checks.
- **Compact representation.** Only d split conditions are stored per tree, compared to up to $2^d - 1$ in a regular tree.
- **Efficient prefix updates.** Because every sample is assigned to a leaf via a fixed binary code, CatBoost can maintain prefix sums of residuals per leaf. This is essential for ordered boosting, where leaf values must be updated incrementally without recomputing the full tree for each prefix.

Although symmetric trees are more restrictive than regular trees, in boosting one does not require each tree to be fully expressive. Instead, the emphasis is on producing many weak but consistent learners. The uniform binary structure makes ordered boosting practical while also improving inference speed and memory usage.

7.3 Leaf Value Updates and Their Role

At each boosting iteration, the tree h_t is fit to approximate the residuals. Each leaf ℓ is assigned a constant value v_ℓ, and the weak learner is defined as

$$h_t(x) = v_{\ell(x)},$$

where $\ell(x)$ is the index of the leaf containing x.

Standard boosting. In ordinary gradient boosting, the leaf value is the average of residuals over the samples that fall into the leaf:

$$v_\ell = \frac{\sum_{k \in L_\ell} r_k^{(t)}}{|L_\ell| + \lambda},$$

with λ as an L2 regularization term. These v_ℓ values together *are* the fitted weak learner h_t. The model is then updated as

$$f_{t+1}(x) = f_t(x) + \eta h_t(x).$$

Ordered boosting. In ordered boosting, we must avoid using a sample's own residual when computing its leaf value. For a sample $\sigma(p)$ at position p in the permutation, the leaf value is instead computed using only earlier samples in the same leaf:

$$v_{\ell,p} = \frac{\sum_{j<p,\, \sigma(j)\in L_\ell} r^{(t)}_{\sigma(j)}}{|\{j < p : \sigma(j) \in L_\ell\}| + \lambda}.$$

Thus each sample's residual is fitted against a leaf value that excludes itself and all later samples, eliminating leakage. Practically, CatBoost implements this efficiently by maintaining prefix sums of residuals and counts per leaf, which can be updated incrementally as the permutation is traversed.

Inheritance across nodes. Each node in the tree represents the subset of samples that satisfy all split conditions along the path from the root to that node. Ordered boosting applies the *global* permutation to the full dataset, and each node or leaf simply inherits the restriction of this order to its own subset. In effect:

- At the root, prefix sums are computed over the entire permutation.
- At each split, the parent's prefix sums are partitioned into left and right subsets according to the split condition.
- No recomputation is required: the children simply continue accumulating statistics for the samples that reach them, respecting their induced order in the global permutation.

This can be understood through the analogy of water flow: the global permutation is a stream of samples, the root node collects the full stream, and each split diverts the flow into left/right channels. Prefix sums are accumulated separately in each channel without reprocessing the entire flow.

Interpretation. In both standard and ordered boosting:

- The leaf values v_ℓ are not merely intermediate statistics; they directly define the weak learner h_t.
- Residuals provide the training signal, but once aggregated into leaf averages (standard) or prefix-averages (ordered), the resulting leaf values are the outputs of the tree.
- The ensemble model is the sum of all such learners: $f_T(x) = f_0(x) + \eta \sum_{t=1}^{T} h_t(x)$.
- Internal nodes (splits) act only as routing devices: they partition the feature space into disjoint subsets. The prediction depends solely on the leaf reached.
- Viewed differently, each tree performs a form of dimension reduction: high-dimensional features are compressed into a categorical leaf index (one of 2^d possible leaves), and the leaf value provides a constant approximation of the residuals for that region.

Why symmetry matters. In regular trees, leaf membership depends on heterogeneous splits, making prefix-based updates in ordered boosting infeasible. Symmetric trees, by contrast, guarantee that leaf assignment is determined by a fixed sequence of global questions, enabling efficient incremental updates.

7.4 Additional Implementation Details

- **Multiple permutations.** CatBoost often uses several random permutations of the training set to reduce variance in ordered statistics.
- **Regularization.** Leaf value estimation includes L2 regularization to prevent overfitting and stabilize updates.
- **Efficient GPU implementation.** The symmetric tree structure and prefix-sum updates are well-suited for GPU parallelization.

7.5 Pseudocode Summary

Algorithm 2 CatBoost Training (Simplified)

Input: dataset $D = \{(x_k, y_k)\}$, iterations T, depth d, learning rate η.
Precompute ordered target statistics for categorical features using permutations.
Initialize $f_0(x) = \text{const}$.
for $t = 1$ to T **do**
  Compute residuals $r_k^{(t)}$ using ordered boosting with prefix models.
  Train symmetric tree h_t of depth d:
    Select splits level by level (same split for all nodes at a level).
    Maintain prefix sums of residuals per leaf.
    Update leaf values incrementally.
  Update model: $f_{t+1}(x) = f_t(x) + \eta h_t(x)$.
end for
Return $f_T(x)$. =0

7.6 Summary of Design Choices

The implementation of CatBoost combines:

- Ordered target statistics for categorical features.
- Ordered boosting for unbiased residual computation.
- Symmetric tree structures for efficient and compact weak learners.
- Incremental leaf value updates to avoid quadratic complexity.

Together, these ensure CatBoost's practical scalability while preserving theoretical guarantees against target leakage and prediction shift.

8 Example Usage